

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

Name: V. Dharani reddy

Reg. No: 192311219

Departement : CSE

AT02-C01-ASSESSMENT : CHAPTER 1

COURSE OUTCOME COVERED

**CO-1: Develop well-structured, standards-compliant web pages
using HTML/XHTML and**

**XML, and apply Internet protocols and HTTP communication
mechanisms for web content**

delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

Code of Conduct:

I **_V Dharani reddy(192311219)**_____ (Name / Reg No)

certify that this submission is my original

work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary

action.

Signature of the student: **V Dharani reddy**

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

QUESTION AND ANSWERS

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

ANSWER:

1. Identify any incorrect or invalid fields in the header.

- **Content-Length: -150** is **invalid** because Content-Length cannot be a negative value.
- The other fields (HTTP/1.1 200 OK, Date, Content-Type, and Connection: keep-alive) are valid.

2. Analyze the issue with Content-Length and explain its impact.

- **Content-Length** specifies the size of the response body in bytes.
- A value of **-150** is invalid because the length cannot be negative.
- **Impact:**
 - Clients may fail to read the response correctly.
 - Browsers may ignore the response or show an error.
 - It can cause communication issues between the client and server.

3. Determine whether the status code matches the response correctness.

- The status code **200 OK** means the request was successful.
- However, the invalid **Content-Length** makes the response incorrect.
- Therefore, the status code does **not fully match** the response because the header contains an error.

Explanation:

- **POST** → Sends form data to the server.
- **/register** → URL where data is sent.
- **Host** → Server address.
- **Content-Type** → Format of submitted data.
- **Body** → Contains the user's name and email.

4. Suggest corrections for the identified errors.

- Replace the invalid **Content-Length: -150** with the correct positive size of the response body.
- **Example:**

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
```

- Ensure that the Content-Length value matches the actual size of the HTML content.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid

ANSWER:

1. DOM Structure Analysis

1. Identify missing or improperly closed elements.

- The `<p>` tag is **not closed** (`</p>` is missing).
- The `<div>` tag is closed before the `<p>` tag, which is an improper nesting.
- The correct closing order should be `</p>` followed by `</div>`.

2. Analyze how the browser will interpret this structure.

- Browsers automatically fix HTML errors.
- The browser will automatically close the `<p>` tag before the `</div>` tag.
- The page will still display **"HelloWorld"**, but the DOM structure will be corrected internally by the browser.

3. Determine the impact on rendering and layout.

- The text **"HelloWorld"** will still be displayed.
- Browsers may create a different DOM structure than intended.
- CSS styling and JavaScript DOM operations may behave unexpectedly due to the corrected structure.

4. Suggest corrections to make the DOM valid.

Use the corrected HTML below:

```
<html><head>
  <title>TestPage</title></head><body>
  <div>
    <p>HelloWorld</p>
  </div></body></html>
</div>
</body>
</html>
```

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

ANSWER:

1. GET vs POST in Login System

1. Interpret the differences in data transmission between GET and POST.

GET

Data is sent in the URL as query parameters.

Data is visible in the browser URL.

Less secure for sensitive data.

POST

Data is sent in the request body.

Data is hidden from the URL.

More secure for sensitive data.

- A normal button only performs actions through JavaScript.
- Without JavaScript or a submit button, the form is never submitted to the server.

2. Analyze which method is more suitable for login and why.

- **POST** is more suitable for login.
- It hides the username and password in the request body.
- It provides better security and prevents credentials from appearing in the URL, browser history, or bookmarks.

3. Identify security risks in using GET for authentication.

- Username and password are visible in the URL.
- Credentials may be stored in browser history and server logs.
- URLs can be shared accidentally, exposing login details.
- Higher risk of credential theft.

4. Suggest best practices for secure login implementation.

- Use **POST** instead of GET for login requests.
- Always use **HTTPS** to encrypt data during transmission.
- Store passwords securely using **hashing**.
- Use strong authentication methods such as **OTP or Multi-Factor Authentication (MFA)**.

Conclusion:

POST is the preferred method for login because it hides sensitive information in the request body and provides better security than **GET**.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

HTML Code Inspection

1. Identify missing closing tags.

- The `<h1>` tag is missing the closing tag `</h1>`.
- The `<p>` tag is missing the closing tag `</p>`.
- The `<img>` tag is a **void element** and does **not** require a closing tag.

2. Analyze issues with the `<img>` tag.

- The `<img>` tag is syntactically correct.
- However, it is missing the **alt attribute**, which is recommended for accessibility.
- If `image.jpg` does not exist, the image will not be displayed.

3. Determine how browsers will handle these errors.

- Browsers automatically correct missing closing tags.
- The page will still display the heading, paragraph, and image.
- However, the DOM structure may differ from the intended HTML, which can affect CSS and JavaScript.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html><html><head>
  <title>Sample</title></head><body>
    <h1>Welcome</h1>
    <p>This is a paragraph</p>
    </body></html>
```

Conclusion:

The HTML has missing `</h1>` and `</p>` closing tags. Adding these tags and including an alt attribute for the `<img>` tag makes the code valid and improves accessibility.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Browser-Server Communication Flow

1. Interpret each step in the communication flow.

1. **User enters a URL** – The user types a website address in the browser.
2. **DNS lookup occurs** – DNS converts the domain name into the server's IP address.
3. **HTTP request is sent** – The browser sends a request to the web server.
4. **Server processes request** – The server handles the request and prepares the response.
5. **Response is returned** – The server sends HTML, CSS, JavaScript, and other resources back to the browser.
6. **Browser renders page** – The browser displays the webpage to the user.

2. Identify where delays can occur.

- Slow **DNS lookup**.
- Network latency while sending the HTTP request.
- Slow server processing due to high traffic or heavy computation.
- Large response size or slow internet connection.
- Browser rendering delays caused by large CSS, JavaScript, or images.

3. Analyze the role of DNS in the process.

- DNS translates the website name (e.g., www.example.com) into an IP address.
- It enables the browser to locate and connect to the correct server.
- Without DNS, users would have to remember IP addresses instead of domain names.

4. Suggest optimizations to improve performance.

- Use **DNS caching** to reduce lookup time.
- Enable **browser caching** for static resources.
- Compress files using **Gzip/Brotli**.
- Use a **Content Delivery Network (CDN)**.
- Optimize images and reduce unnecessary JavaScript/CSS files.

Conclusion:

The browser-server communication flow involves DNS lookup, HTTP request/response, and page rendering. Performance can be improved through caching, compression, CDNs, and optimized resources.

窗体顶端

窗体底端

-